

Executive Architectural Blueprint: MediKiosk Development and AI-Native Integration Strategy

Phase 1: Project Deconstruction and AI-First Technology Stack Rationale

The development of MediKiosk represents a highly complex intersection of public health infrastructure, natural language processing, and high-throughput data architecture. In the heavily congested Outpatient Departments (OPDs) of Indian government hospitals, physicians face extraordinary volume, often consulting between 4,000 and 10,000 patients daily¹. This volume severely constricts the diagnostic window to a mere two to five minutes per patient, precipitating a crisis of incomplete medical histories, overlooked comorbidities, and systemic diagnostic vulnerability¹. The complexity is magnified exponentially within the AYUSH (Ayurveda, Yoga & Naturopathy, Unani, Siddha, and Homeopathy) medical frameworks, which demand intricate historical assessments encompassing a patient's fundamental constitution (Prakriti), metabolic capacity (Agni), and lifestyle metrics (Ahara-Vihara)¹.

To engineer a patient-facing, multi-lingual kiosk capable of autonomously capturing this data before the physician consultation, the system architecture must be meticulously deconstructed into four primary operational modules. Because the primary developer of this codebase will be a Large Language Model (LLM), each module must be evaluated not through the lens of human developer ergonomics, but through the lens of "AI Generation Accuracy"—defined by structural determinism, abstract syntax tree (AST) uniformity, and syntactic explicitness.

The first module is the Conversational History Engine. This subsystem must function as an autonomous clinical interviewer, interacting via voice or touch interfaces in English, Hindi, and Tamil¹. It must process noisy audio environments, execute dynamic stateful questioning (e.g., following a chest pain claim with standard SOCRATES assessment vectors), and perform real-time triage to flag emergency conditions like dyspnoea for immediate clinical escalation¹. The second module is the Medical Document Intelligence pipeline. Patients frequently present with disorganized, physically fragmented records spanning handwritten prescriptions and printed lab reports¹. This subsystem requires Optical Character Recognition (OCR) capable of ingesting varied physical media, normalizing the extracted entities, and structuring the data into a chronological clinical timeline while actively monitoring for contraindications¹.

The third module is the Structured Summary Generator. This operates as a stateless data transformation engine that synthesizes the unstructured conversational transcripts and the structured OCR chronologies into a highly formalized, single-page medical summary. This payload must be rendered simultaneously in the patient's local language for verification and in English or Hindi for the physician's review¹.

The fourth module comprises the Privacy, Consent, and Integration layer, explicitly interfacing with the Ayushman Bharat Digital Mission (ABDM). This entails secure OAuth 2.0 gateway handshakes, Ayushman Bharat Health Account (ABHA) identity verification, and the orchestration of Health Information Provider (HIP) and Health Information User (HIU) interoperability standards utilizing the Fast Healthcare Interoperability Resources (FHIR) R4 specifications¹.

Evaluating technology stacks for these modules requires prioritizing frameworks that mitigate LLM hallucinations. LLMs demonstrate severe degradation in reasoning when forced to navigate frameworks burdened with "magic"—dynamic metaprogramming, implicit dependency injection, or extensive annotation-driven configurations. When logic is hidden behind framework abstractions rather than declared explicitly in the code, the AI is forced to guess the runtime behavior, leading to catastrophic integration failures.

Technology Stack Option	Primary Languages	Evaluation for AI-Native Generation	Generation Accuracy Verdict
Enterprise Java Ecosystem	Java, XML	Utilizes Spring Boot and React/Redux. Relies heavily on reflection, @Autowired dependency injection, and implicit Bean lifecycles. LLMs struggle to map cross-file state mutations in Redux and frequently hallucinate Proxy behaviors in Spring Data JPA.	Rejected: High probability of hallucination due to implicit configuration and excessive boilerplate scattering context.
Modern Python API Stack	Python, TypeScript	Utilizes FastAPI, SQLAlchemy 2.0, and Next.js. FastAPI provides excellent OpenAPI generation. However, Python's fundamentally	Suboptimal: Strong analytical capabilities, but the language boundary between backend and frontend introduces unacceptable API

		dynamic nature requires complex boundary typing (Pyright/Pydantic) that must be manually synchronized with the TypeScript frontend, creating a seam where AI frequently hallucinates type mismatches.	contract drift during rapid AI generation.
Unified TypeScript Monorepo	TypeScript	Utilizes Next.js (App Router), Hono.js, and Drizzle ORM. Delivers 100% end-to-end static typing. Employs explicit control flow, shared Zod validation schemas across the stack, and utilizes standard Web APIs. The AST remains uniform from database schema to UI component.	Winner: Maximum deterministic generation. The compiler prevents compounding logic errors, and the AI maintains a continuous syntactic context.

The selection of the unified TypeScript monorepo (Next.js, Hono.js, Drizzle ORM) is justified across four critical evaluation metrics. Regarding free-tier viability, Hono.js is an edge-native framework designed for deployment on serverless architectures like Cloudflare Workers, which offer immense free execution tiers well suited for hackathon environments⁴. Runtime performance is exceptional, as Hono achieves sub-millisecond overhead, which is critical for minimizing latency in the voice-to-text Conversational History Engine. In terms of LLM training data density, TypeScript, React, and Zod are among the most heavily represented syntaxes in the training corpora of modern foundational models, ensuring high-fidelity code generation. Finally, regarding error message clarity, the strict TypeScript

compiler (tsc) and Zod runtime validators produce hyper-specific fault localization. When an error occurs, the compiler outputs exact diagnostic strings (e.g., Property 'Prakriti' is missing in type...). Feeding this exact deterministic string back into the AI's context window results in a near 100% resolution rate on the first retry, establishing a zero-hallucination debugging loop.

Flagging Missing Technical Details: While the TypeScript monorepo is declared the winner, the project description does not explicitly state whether the frontend requires Server-Side Rendering (SSR) for Search Engine Optimization, or if a statically generated Single Page Application (SPA) delivered via Edge CDN is preferred. Given the kiosk deployment model, Static Site Generation (SSG) with client-side hydration is assumed for maximum resilience on unstable hospital networks, but this will be confirmed in the final clarifying questions.

Phase 2: The Complete Free Toolchain (AI-Optimized)

To execute the winning architecture, the toolchain must be restricted to libraries that minimize prompt complexity. Every selected tool enforces strict boundaries, preventing the LLM from inventing logic or deviating from safe software patterns.

The frontend framework is Next.js utilizing the App Router. The App Router enforces a strict file-system-based routing convention. LLMs map file-system routing perfectly because the relationship between the URL path and the component logic is explicitly defined by the folder structure, eliminating the need to generate complex routing configuration objects. For state management, Zustand is mandated over Redux. Redux fragments application state across actions, reducers, and selectors, which rapidly exhausts the AI's context window and leads to logic drift. Zustand consolidates state into a single, predictable, hook-based block, allowing the AI to observe and modify the entire state lifecycle within a single prompt iteration.

The UI component library of choice is shadcn/ui combined with Tailwind CSS. Traditional libraries like Material UI or Ant Design distribute pre-compiled NPM packages, hiding the underlying DOM structure. When an AI is asked to modify a complex interactive element in those libraries, it often hallucinates non-existent props or CSS overrides. Conversely, shadcn/ui copies raw, statically typed React code directly into the project directory. The AI modifies local, explicit code, drastically reducing hallucinations regarding library-specific configurations.

For the backend framework, Hono.js combined with @hono/zod-openapi is selected. While FastAPI is traditionally favored for automated Swagger documentation, Hono achieves identical OpenAPI 3.1 specification generation natively in TypeScript. By defining a Zod schema once, Hono automatically enforces runtime request validation, generates the corresponding TypeScript types for the application, and writes the OpenAPI JSON specification. This eliminates redundant logic generation, ensuring the AI cannot create a discrepancy between the validation logic and the API documentation.

The database interaction layer relies entirely on Drizzle ORM, specifically rejecting Prisma. Prisma utilizes a proprietary .prisma schema language and a Rust-based query engine that requires an intermediate compilation step. LLMs occasionally misconfigure this proprietary syntax or fail to anticipate the behavior of the generated Prisma Client proxy objects. Drizzle ORM defines schemas directly in standard TypeScript, mirroring standard SQL syntax closely⁴. This ensures the AI remains anchored in a single, predictable syntax environment, eliminating SQL injection vectors while maintaining type safety.

Data validation across the entire stack is governed by Zod. Zod is structurally critical for the Conversational History Engine. The AI will be instructed to coerce all unstructured output from the LLM-driven patient interviews into strict Zod schemas. If the patient's narrative omits a critical diagnostic detail, Zod will throw a deterministic error. The application logic catches this error and instructs the voice agent to generate a targeted follow-up question, entirely removing the need for the AI to write manual, deeply nested if/else validation trees. Authentication and authorization are managed by Clerk. Clerk provides highly secure, pre-built React components and explicit Next.js middleware. Implementing security protocols from scratch invites catastrophic AI hallucinations regarding cryptographic hashing, session management, and cross-site scripting protections. By utilizing Clerk, the AI only needs to write declarative route-protection wrappers, confining its operational scope to simple boolean checks rather than complex security engineering.

Phase 3: The "Free API Keys" Master List

The MediKiosk prototype relies on zero-cost, high-allowance external APIs to manage complex machine learning inferences and database hosting. To prevent the AI from generating hardcoded credentials or hallucinating variable names, the following services must be registered, and their corresponding keys stored in an explicitly structured `.env.example` file.

Service Category	Provider Platform	Free Tier Quota and Operational Details	Direct Sign-up URL
Speech-to-Text & TTS (Indic Languages)	Sarvam AI	Provides ₹100 to ₹1,000 in free API credits upon sign-up ⁵ . The Bulbul v3 TTS and STT APIs natively support English, Hindi, and Tamil, gracefully handling intra-sentence code-switching (e.g., Hinglish) without requiring manual boundary detection ⁷ . Billing equates to approximately ₹30 per hour of transcribed audio ⁸ .	https://sarvam.ai/

Core LLM Orchestration	Groq API	Delivers exceptionally high free rate limits for open-weight models such as Llama 3 ⁹ . Groq's specialized Tensor Streaming Processor architecture guarantees the sub-second inference latency required for a natural conversational voice agent.	https://console.groq.com/
Document OCR (Handwriting Analysis)	Google AI Studio (Gemini)	Offers a highly generous free tier for developers ⁹ . The Gemini 1.5 Flash multimodal model is exceptionally proficient at extracting structured entity data from poorly illuminated or heavily cursive handwritten medical prescriptions.	https://aistudio.google.com/
Health Network Integration	NHA ABDM Sandbox	Grants free, unmetered access to the Milestone 1, 2, and 3 testing environments ¹¹ . Provides mock ABHA identities,	https://sandbox.abdm.gov.in/

		cryptographic certificates, and FHIR gateway routing for testing consent management without exposing real protected health information ² .	
Serverless Database Compute	Neon PostgreSQL	Provisions 500 MB of storage with compute that automatically scales to zero when idle. Integrates flawlessly with Drizzle ORM's migration utilities without requiring complex Virtual Private Cloud networking.	https://neon.tech/

AI Context Loading Note: The developer must create an .env.example file containing the precise keys SARVAM_API_KEY, GROQ_API_KEY, GEMINI_API_KEY, NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY, CLERK_SECRET_KEY, DATABASE_URL, and ABDM_CLIENT_SECRET. This file must be fed into the AI's context before generating backend logic to establish a rigid nomenclature for environment variables.

Phase 4: 100% Free Hosting Architecture

The deployment environment must be strategically selected based on configuration simplicity. Orchestration systems relying on Kubernetes YAML or Terraform HCL introduce excessive operational complexity, leading to AI hallucinations regarding ingress controllers, persistent volume claims, and pod lifecycles. Therefore, the architecture relies exclusively on Edge and Serverless functions configured via simple JSON or TOML files.

The compute platform will utilize Cloudflare Pages for the statically exported Next.js frontend and Cloudflare Workers for the Hono.js backend. Cloudflare environments are configured using a minimalist wrangler.toml file⁴. Deploying to edge workers forces a paradigm of stateless, single-purpose function execution. If an AI generates a memory leak or a blocking synchronous process, the failure is isolated to that specific ephemeral execution rather than accumulating within a persistent Node.js virtual machine and causing systemic application crashes.

Data persistence is delegated to Neon Serverless Postgres. Neon connects via a standard

connection string over HTTPS, allowing Drizzle ORM to execute schema migrations directly from a GitHub Actions pipeline or local terminal without requiring Dockerized database networking. For file storage—specifically the temporary ingestion of patient medical documents for OCR processing—the system will utilize Cloudflare R2. R2 is fully S3-compatible, offers a free tier of 10 gigabytes per month, and eliminates data egress fees, which is critical for transmitting image blobs between the kiosk, the edge worker, and the Gemini API.

Phase 5: The "AI Coding Toolkit" (Prompt Engineering Setup)

To enforce strict, zero-hallucination output, the Integrated Development Environment (IDE) must be architecturally weaponized to restrict the AI's creative liberty regarding syntax and error handling.

The project root must contain a `.cursorsrules` (or equivalent AI instruction file) to globally govern the AI's generation behavior. The following JSON configuration dictates that the AI must never utilize the any type, must declare explicit return boundaries, and cannot silently swallow runtime exceptions:

```
JSON
{
  "rules":
  {
    "typescript":
    {
      "strict": true
      "noImplicitAny": true
      "explicitReturnType": true
      "forceZodValidation": "All incoming REST and RPC payloads MUST be strictly validated with Zod before processing."
    }
    "drizzle":
    {
      "banRawSQL": "Never use db.execute() with raw string templates. You are strictly confined to Drizzle relational queries and typed query builders to prevent SQL injection."
    }
    "errorHandling":
    {
      "noSilentFailures": "Do not utilize empty try/catch blocks. All caught errors must be propagated or logged via console.error with a highly descriptive contextual trace."
    }
  }
}
```

The recommended underlying foundational model for this specific stack is Claude 3.5 Sonnet.

Extensive analysis indicates that Claude 3.5 Sonnet significantly outperforms competitor models (including GPT-4o) in structural adherence to modern Next.js App Router conventions and Drizzle ORM schema definitions, demonstrating superior contextual recall over extended code-generation sessions.

Before initiating any sprint, the AI must be primed with a Base System Prompt to contextualize its operational parameters. The developer must supply the following instruction: "You are an expert TypeScript architect engineering MediKiosk, a highly regulated clinical history platform. You strictly adhere to the Next.js App Router, Hono.js, Drizzle ORM, and Zod. You do not invent proprietary frameworks. You never write an any type. If a business requirement is ambiguous or missing, you must write a discrete function signature returning a TODO error state rather than guessing complex clinical logic."

Crucially, contextual loading must precede business logic generation. Before asking the AI to write routing logic, the developer must feed package.json, tsconfig.json, and drizzle.config.ts into the AI's context window. This forces the LLM to analyze the exact versions of the installed dependencies, preventing it from utilizing deprecated syntaxes or generating logic that violates the established TypeScript compiler strictness.

Phase 6: Critical Sources & "Anchor" Files

To prevent the AI from defaulting to generic, outdated Express.js templates commonly found in its training data, the developer must provide "Anchor Files." These are verified, production-grade files from successful open-source repositories that set the syntactical standard for the project. The AI will map its generation logic directly against these anchors. The developer must reference the eric-sison/nextjs-honojs-boilerplate repository on GitHub¹³. This repository is a verified implementation of the exact Next.js, Hono, and Drizzle stack required.

- **Anchor Action 1:** Extract the Hono mounting structure. Provide the AI with the boilerplate's route.ts file and explicitly instruct: "Review how Hono RPC is mounted within this Next.js App Router architecture. Generate our app/api/[...route]/route.ts file identically, mounting our API utilizing the edge runtime."
- **Anchor Action 2:** Extract the database configuration. Copy the drizzle.config.ts file from the repository and instruct the AI: "Copy the drizzle.config.ts from this repo exactly. Do not invent alternative database connection parameters or mapping configurations."
- **Anchor Action 3:** Establish the ABDM FHIR standard. Navigate to the NHA ABDM Sandbox documentation regarding Milestone 3 Data Exchange³. Extract a sample JSON payload for a FHIR R4 DiagnosticReport³. Instruct the AI: "When engineering the structured summary generator, the resulting payload must conform strictly to this exact HL7 FHIR R4 JSON schema to ensure successful interoperability with the Ayushman Bharat Digital Mission."

Phase 7: The Step-by-Step Execution Plan (Zero-Hallucination Strategy)

The architectural build is partitioned into deterministic sprints. Following every generation

request, the developer must execute a strict terminal command to validate the AST and compilation state. If the validation command fails, the output error string must be fed directly back to the AI. Under no circumstances should the developer proceed to the next sprint if the validation command fails; doing so will cause hallucinations to compound exponentially.

Sprint Focus	Prompt to Copy-Paste to the AI	Validation Command
Sprint 1: Schema & ORM Foundation	"Read the MediKiosk requirements. Create a Drizzle ORM schema in db/schema.ts. Define tables for patients (demographics and ABHA ID), sessions (kiosk visits), conversations (JSONB arrays of voice transcripts), and summaries (final structured clinical output). Use UUIDs for primary keys. Generate the corresponding Zod insertion schemas using drizzle-zod."	<code>npx tsc --noEmit && npx drizzle-kit generate</code>
Sprint 2: Backend Core API	"Utilizing the Drizzle schema from Sprint 1, construct a Hono.js router in app/api/[...route]/route.ts. Establish endpoints for POST /api/sessions and PATCH /api/sessions/:id. You must use @hono/zod-validator to intercept and validate the incoming request body against the Zod schemas from Sprint 1 before executing database operations. Return strictly typed JSON responses."	<code>npx tsc --noEmit</code>

Sprint 3: AI Voice & Triage Engine	"Engineer a service layer for the Conversational Engine. Write a function that transmits an audio blob to the Sarvam AI Speech-to-Text API and returns the transcript¹⁴. Next, write a function that routes this transcript to the Groq API utilizing the LLaMA 3 model. Construct a Groq system prompt to evaluate the transcript for emergency 'Red Flag' symptoms (e.g., dyspnoea). The Groq output must be coerced into a strict Zod-validated JSON object indicating if triage is required and formulating the next follow-up question¹."	<pre>npx tsc --noEmit && npm run lint</pre>
Sprint 4: ABDM Sandbox Integration	"Implement the ABDM Milestone 1 and Milestone 2 gateway authentication flows¹¹. Write a stateless utility function that utilizes the ABDM_CLIENT_SECRET to retrieve an OAuth 2.0 gateway session token from /api/hiecm/gateway/v3/sessions². Then, write a transformation function that maps our summaries database row into an HL7 FHIR R4 standard JSON payload suitable for ABDM health record linking³."	<pre>npx tsc --noEmit</pre>

Sprint 5: Frontend UI Assembly	"Generate a Next.js frontend utilizing shadcn/ui. The interface must be heavily icon-based and optimized for non-tech-savvy users on touch-screen kiosks¹. Implement a Zustand store to manage the ephemeral state of the patient's active session. Construct a prominent 'Hold to Speak' interface leveraging the standard MediaRecorder API. Ensure all data fetching utilizes the Hono RPC client types so the communication between the UI and API is perfectly type-safe."	<pre>npm run build</pre>
---------------------------------------	---	--------------------------

Phase 8: Free-Tier Disaster Recovery

Relying exclusively on free-tier application programming interfaces necessitates a robust, automated disaster recovery architecture to prevent catastrophic kiosk failures in a high-throughput, mission-critical hospital environment. The system must possess self-healing and failover capabilities.

If the Sarvam AI Speech-to-Text API exhausts its free credit allocation (resulting in a 429 Rate Limit Exceeded HTTP status code)⁵, the system must automatically degrade to utilizing the Web Speech API, natively supported within the kiosk's Chromium-based browser. While the native browser API demonstrates inferior accuracy when parsing complex Hindi and Tamil code-switching compared to Sarvam's custom models⁷, it provides a zero-cost, infinite-capacity fallback to ensure operations do not cease.

In the event of severe rate limiting on the Groq API for LLaMA 3 inference⁹, the application logic must execute a graceful failover to the Google Gemini API free tier¹⁰. Because the core LLM orchestration logic is abstracted behind a generic service interface utilizing standardized Zod schemas for output parsing, the business logic remains entirely agnostic to the underlying foundational model.

Finally, should the Neon serverless database experience connection drops due to edge latency spikes or cold start timeouts, the Next.js frontend (via the Zustand state manager) must temporarily cache the patient's ongoing conversational transcript and OCR payload within the browser's persistent IndexedDB or localStorage. Upon restoration of network connectivity, a background synchronization utility will silently flush the local queue to the Hono API, ensuring

zero data loss during localized internet outages.

This is for informational purposes only. For medical advice or diagnosis, consult a professional.

Clarifying Questions

To achieve maximum determinism and refine the precise architectural parameters for the subsequent phase of boilerplate generation, I require answers to the following technical specifications. Please review and select the appropriate response for each:

1. Regarding the Conversational History Engine's audio processing, which approach optimally aligns with your hardware and network constraints?

- **A) Streaming Audio:** Audio is streamed to the Sarvam API via WebSockets for real-time latency (<250ms), which provides a natural conversational cadence but demands highly robust frontend state management.
- **B) Chunked Uploads:** The patient completes their statement, and a single .wav payload is transmitted via REST API, slightly increasing response latency but maximizing codebase simplicity.
- **C) Edge Processing:** Fundamental wake-word detection executes locally in the browser via WebAssembly before transmitting heavy payloads to the cloud to preserve API credits.

2. How must the system manage ABDM (Ayushman Bharat Digital Mission) compliance specifically for low-literacy patients lacking smartphones?

- **A) Assisted Mode:** The kiosk UI provides a secondary administrative workflow for hospital staff to input the patient's Aadhaar demographic data to generate an ABHA identity on the spot.
- **B) Biometric Verification:** The kiosk hardware integrates a fingerprint or iris scanner (which necessitates local hardware drivers outside the scope of standard web APIs).
- **C) Optional ABDM Linkage:** ABDM linkage remains strictly optional; patients are permitted to proceed anonymously or linked only by a localized hospital token.

3. For the Medical Document Intelligence (OCR) module, what is the anticipated volume of physical document uploads per patient?

- **A) Low Volume (1-2 pages):** Synchronous processing via the Gemini Flash API is acceptable while the patient actively waits at the kiosk screen.
- **B) High Volume (10+ pages):** Requires an asynchronous background queuing architecture, allowing the patient to proceed to the physician while documents are routed to the doctor's dashboard post-processing.
- **C) Zero Physical Documents:** The platform will rely exclusively on the ABDM digital fetch (M3 module) for historical records, bypassing OCR entirely.

4. The software must interface in English, Hindi, and Tamil. How should the user interface text elements (buttons, instructional prompts) be managed?

- **A) Static Translation Dictionaries:** Utilizing next-intl or i18next with pre-defined, hardcoded JSON dictionaries for all visual UI text.
- **B) Dynamic LLM Translation:** The UI text is generated dynamically by the Groq API based on the selected language to facilitate highly contextual prompts based on the patient's ongoing narrative.

- **C) Audio-Only Guidance:** The UI remains visually static in English (utilizing oversized iconography), while the Sarvam TTS engine provides all instructional guidance audibly in Hindi or Tamil.

5. What is the mandatory deployment environment for the final Hackathon presentation and prototype demonstration?

- **A) Fully Cloud Serverless:** Hosted exclusively on Cloudflare Workers with Neon PostgreSQL to optimally demonstrate cloud scalability.
- **B) Localized / Dockerized:** Assuming unstable hospital internet availability, the entire stack (excluding external LLM APIs) must be capable of executing via docker-compose up on a local development machine.
- **C) Hybrid Edge Infrastructure:** The frontend is delivered via Cloudflare Edge CDN, but the PostgreSQL database must run locally on a simulated hospital server to demonstrate localized data compliance.

Works cited

1. Patient Case-Taking Software - Project Overview.pdf
2. ABDM Sandbox API Documentation | PDF | Identifier - Scribd, <https://www.scribd.com/document/868017730/M3-Dcoument-03-03-2025-faf0c9aecbSandbox>
3. ABDM FHIR Integration Guide 2026: How to Make Your ... - Adrine, <https://www.adrine.in/blog/abdm-fhir-integration-guide-2026>
4. drizzle · GitHub Topics, <https://github.com/topics/drizzle>
5. Credits & Rate Limits | Sarvam API Docs, <https://docs.sarvam.ai/api/getting-started/ratelimits>
6. Sarvam AI: ₹1,000 free API credits (22 Indian languages STT/TTS), <https://aicredits.dev/submissions/180-sarvam-ai-1-000-free-api-credits-22-indian-languages-stt-tts-translation>
7. Text to Speech & AI Voice Generator for Indian Languages - Sarvam AI, <https://www.sarvam.ai/text-to-speech>
8. Pricing | Sarvam API Docs, <https://docs.sarvam.ai/api/getting-started/pricing>
9. Best Free AI APIs in 2026: Ten Endpoints With Rate Limits That, <https://singularitybyte.com/tools/best-free-ai-apis-generous-rate-limits-2026.html>
10. 5 Best Free AI APIs for Developers in 2026 (No Credit Card Required), <https://medium.com/@christophorusdryantoro/5-best-free-ai-apis-for-developers-in-2026-no-credit-card-required-9d5ef3ea6a4a>
11. ABDM M1 M2 M3 API Development for Hospitals - DigiEn InfoSoft LLP, <https://www.digien.in/digien-abdm.html>
12. ABDM Sandbox Documentation, <https://kiranma72.github.io/abdm-docs/>
13. A full-stack boilerplate built with Next.js and Hono.js ... - GitHub, <https://github.com/eric-sison/nextjs-honojs-boilerplate>
14. Speech to Text API for Indian Languages - Sarvam AI, <https://www.sarvam.ai/apis/speech-to-text>